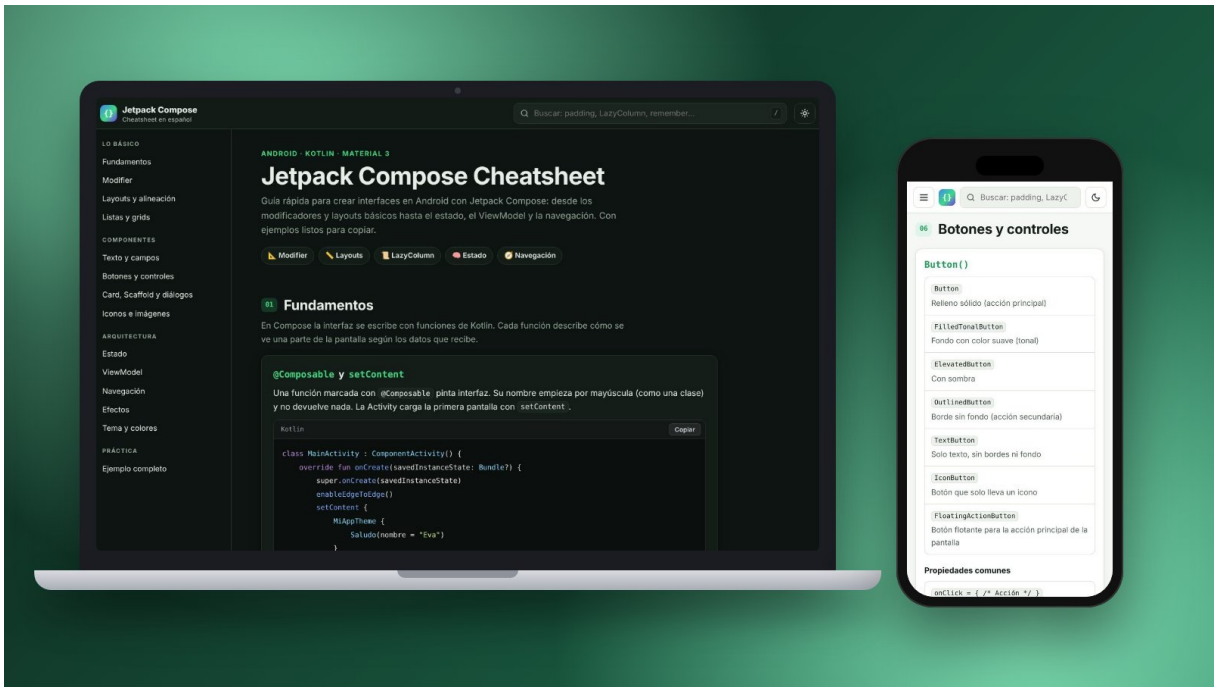


Jetpack Compose Cheatsheet

Guía rápida en español de Jetpack Compose y Material 3

Eva Gallardo Romero

3 de marzo de 2026

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **2**

4 Requisitos del sistema **3**
4.1 Requisitos funcionales 3
4.2 Requisitos no funcionales 4

5 Documentación técnica **4**
5.1 1. Tema 4
5.2 3. Bloques de código 4
5.3 4. Menú lateral activo 5
5.4 5. Buscador 6

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: Jetpack-Compose-Cheatsheet.

La guía está publicada en evagr11.github.io/Jetpack-Compose-Cheatsheet y la documentación del código generada con JSDoc, en evagr11.github.io/Jetpack-Compose-Cheatsheet/docs.

1 Descripción general

Jetpack Compose Cheatsheet es una **guía rápida en español** para crear interfaces Android con **Jetpack Compose** y **Material 3**. Cubre lo necesario para hacer una app completa: desde el primer `@Composable` hasta el estado, el `ViewModel` y la navegación.

Empezó como mis apuntes de modificadores y layouts. Al usarlos en proyectos como mi TFC iMirly se quedaron cortos, así que los amplié y los pasé a una **web**, que se consulta mejor que un README largo. Son a la vez una web y unos apuntes.

- **14 secciones y 44 fichas**: fundamentos, Modifier, layouts y alineación, listas y grids, texto y campos, botones y controles, Card y Scaffold, iconos e imágenes, estado, ViewModel, navegación, efectos, tema y un ejemplo completo.
- **Buscador instantáneo** que no tiene en cuenta tildes ni mayúsculas, resalta las coincidencias y se abre con la tecla `/`.
- **29 bloques de código Kotlin** con resaltado de sintaxis hecho a mano y botón *Copiar*.
- **Modo claro y oscuro**: sigue el del sistema y recuerda la elección.
- **Responsive**: en el móvil el menú pasa a un panel lateral.

Está hecha en **HTML, CSS y JavaScript** sin librerías y se publica en **GitHub Pages**.

2 Diagrama de clases

No hay clases: `index.html` tiene todo el contenido y `main.js` le añade la interacción en cinco bloques.

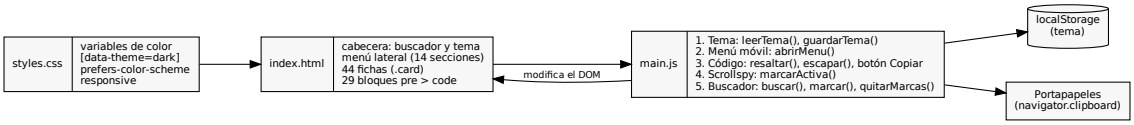
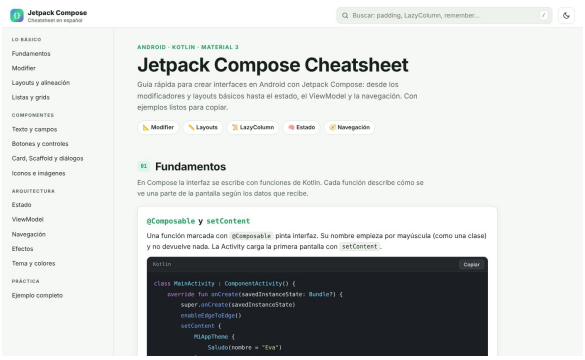


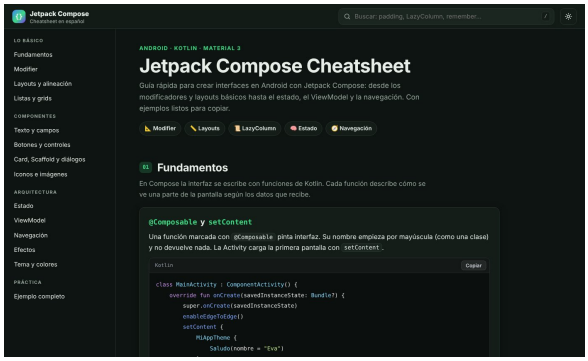
Figura 1: Estructura de la Cheatsheet

3 Diseño

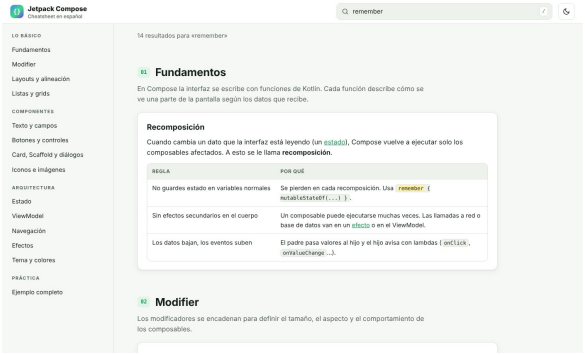
Una cabecera fija con el nombre, el buscador y el botón del tema; un menú lateral con las secciones agrupadas (Lo básico, Componentes, Arquitectura y Práctica) que marca la sección que se está leyendo; y el contenido en fichas blancas con tablas, avisos de errores típicos y bloques de código oscuros. El verde de Android es el color de acento.



Modo claro



Modo oscuro



Búsqueda de «remember»



Móvil

Figura 2: Capturas de la Cheatsheet

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	La guía debe organizarse en **secciones** con un menú lateral que lleve a cada una.
RF02	El menú debe marcar la sección que se está leyendo al hacer scroll.
RF03	El **buscador** debe ocultar las fichas que no contienen el texto, sin tener en cuenta tildes ni mayúsculas, y resaltar las coincidencias.
RF04	Debe indicar cuántos resultados hay o avisar si no hay ninguno.
RF05	La tecla `/` debe abrir el buscador y `Escape`, vaciarlo.
RF06	Los bloques de código deben verse **resaltados** y tener un botón para **copiarlos**.
RF07	Debe haber **modo claro y oscuro**, siguiendo el del sistema y recordando la elección.
RF08	En el móvil el menú debe abrirse como panel lateral y cerrarse al elegir una sección, pulsar fuera o `Escape`.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	Hecha en **HTML, CSS y JavaScript** sin librerías externas, ni siquiera para el resaltado.
RNF02	Publicada en **GitHub Pages**.
RNF03	El código se escapa antes de resaltarlo para no inyectar HTML.
RNF04	El scroll se procesa una vez por fotograma (`requestAnimationFrame`) para no ralentizar la página.
RNF05	Si el navegador no deja guardar ni copiar (modo privado), la web sigue funcionando.
RNF06	El código debe tener comentarios **JSDoc**.

5 Documentación técnica

Este anexo agrupa la documentación por bloques de `main.js`, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado.

5.1 1. Tema

5.1.1 Ubicación

`js/main.js`

Descripción general:
Guarda la elección de tema; si no hay, manda la del sistema.

5.1.2 Funciones

Botón del tema **Descripción:** Calcula si ahora está en oscuro (por la elección guardada o por el sistema) y cambia al contrario.

Código comentado:

```
function leerTema() {
  try { return localStorage.getItem('tema'); } catch (e) { return null; }
}

const temaGuardado = leerTema();
if (temaGuardado) root.dataset.theme = temaGuardado;

document.getElementById('theme').addEventListener('click', () => {
  const oscuroAhora = root.dataset.theme
    ? root.dataset.theme === 'dark'
    : window.matchMedia('(prefers-color-scheme: dark)').matches;
  const nuevo = oscuroAhora ? 'light' : 'dark';
  root.dataset.theme = nuevo;
  guardarTema(nuevo);
});
```

5.2 3. Bloques de código

5.2.1 Ubicación

`js/main.js`

Descripción general:
Resaltado de Kotlin con una expresión regular y botón para copiar.

5.2.2 Funciones

resaltar **Descripción:** Recorre el código con una expresión regular que reconoce comentarios, cadenas, anotaciones, números y palabras, y envuelve cada uno en un `span` con su clase. Lo que no coincide se escapa.

Código comentado:

```
const KEYWORDS = new Set(('fun val var if else when for while return by class data object sealed interface ' +
  'private override import package in is as true false null it this suspend').split(' '));
const TOKEN = /(\/\/.*$|\/\*[\s\S]*?\*\/)|("(?:\\.|[^"\\])*")|(@[A-Za-z]+)|(\b\d+(?:\.\d+)?f?\b)|(\b[A-Za-z_][A-Za-z0-9_]*\b)/gm;

function resaltar(codigo) {
  let html = '';
  let ultimo = 0;
  codigo.replace(TOKEN, (match, comentario, cadena, anotacion, numero, palabra, pos) => {
    html += escapar(codigo.slice(ultimo, pos));      // lo que hay entre tokens
    ultimo = pos + match.length;
    let clase = '';
    if (comentario) clase = 'c';
    else if (cadena) clase = 's';
    else if (anotacion) clase = 'a';
    else if (numero) clase = 'n';
    else if (KEYWORDS.has(palabra)) clase = 'k';
    // seguida de ( o {: función (minúscula) o tipo (mayúscula)
    else if (/^\s*[({]/.test(codigo.slice(ultimo))) clase = /^[A-Z]/.test(palabra) ? 't' : 'f';
    else if (/^[A-Z]/.test(palabra)) clase = 't';
    html += clase ? `<span class="${clase}">${escapar(match)}</span>` : escapar(match);
    return match;
  });
  return html + escapar(codigo.slice(ultimo));
}
```

Botón Copiar **Descripción:** Copia el código al portapapeles y lo confirma durante 1,6 segundos. Si el navegador no lo permite, pide copiarlo a mano.

Código comentado:

```
boton.addEventListener('click', async () => {
  try {
    await navigator.clipboard.writeText(texto);
    boton.textContent = '¡Copiado!';
    boton.classList.add('ok');
  } catch (e) {
    boton.textContent = 'Selecciona y copia';
  }
  setTimeout(() => { boton.textContent = 'Copiar'; boton.classList.remove('ok'); }, 1600);
});
```

5.3 4. Menú lateral activo

5.3.1 Ubicación

`js/main.js`

Descripción general:
Scrollspy: marca en el menú la sección que se está leyendo.

5.3.2 Funciones

marcarActiva **Descripción:** La sección activa es la última cuyo título ya ha pasado por debajo de la cabecera. Se calcula como mucho una vez por fotograma.

Código comentado:

```
function marcarActiva() {
  pendiente = false;
  let actual = null;
  seccionesMenu.forEach((s) => {
    if (!s.classList.contains('hidden') && s.getBoundingClientRect().top <= 140)
        actual = s.id;
  });
  enlaces.forEach((a, id) => a.classList.toggle('active', id === actual));
}

window.addEventListener('scroll', () => {
  if (!pendiente) { pendiente = true; requestAnimationFrame(marcarActiva); }
}, { passive: true });
```

5.4 5. Buscador

5.4.1 Ubicación

`js/main.js`

Descripción general:
Filtra las fichas y resalta lo encontrado.

5.4.2 Funciones

normalizar **Descripción:** Quita las tildes y pasa a minúsculas para comparar.

Código comentado:

```
const normalizar = (t) => t.normalize('NFD').replace(/[\u0300-\u036f]/g, '').
    toLowerCase();
tarjetas.forEach((t) => { t.dataset.busqueda = normalizar(t.textContent); });   // se
    calcula una vez
```

buscar **Descripción:** Oculta las fichas sin coincidencias y las secciones vacías, resalta lo encontrado y escribe cuántos resultados hay.

Código comentado:

```
function buscar() {
  const termino = normalizar(input.value.trim());
  quitarMarcas();
  document.body.classList.toggle('searching', termino.length > 0);
  if (!termino) {
    tarjetas.forEach((t) => t.classList.remove('hidden'));
    secciones.forEach((s) => s.classList.remove('hidden'));
    vacio.style.display = 'none';
    return;
  }
  let encontrados = 0;
  tarjetas.forEach((t) => {
    const coincide = t.dataset.busqueda.includes(termino);
    t.classList.toggle('hidden', !coincide);
    if (coincide) { encontrados++; marcar(t, termino); }
  });
  secciones.forEach((s) => s.classList.toggle('hidden', !s.querySelector('.card:not(.
      hidden)')));
  info.textContent = `${encontrados} ${encontrados === 1 ? 'resultado' : 'resultados'}
      para «${input.value.trim()}»`;
  vacio.style.display = encontrados ? 'none' : 'block';
}
```

marcar **Descripción:** Recorre los nodos de texto de la ficha (sin entrar en el código) y envuelve en `<mark>` la primera coincidencia de cada uno.

Código comentado:

```
function marcar(elemento, termino) {
  const recorrido = document.createTreeWalker(elemento, NodeFilter.SHOW_TEXT, {
    acceptNode: (n) => (n.parentElement.closest('pre, .code-head') ? NodeFilter.
        FILTER_REJECT : NodeFilter.FILTER_ACCEPT),
  });
  const nodos = [];
  while (recorrido.nextNode()) nodos.push(recorrido.currentNode);
  nodos.forEach((nodo) => {
    const texto = nodo.textContent;
    const i = normalizar(texto).indexOf(termino);
    if (i === -1) return;
    const mark = document.createElement('mark');
    mark.textContent = texto.slice(i, i + termino.length);
    const resto = nodo.splitText(i);                          // corta el texto en dos
    resto.textContent = resto.textContent.slice(termino.length);
    nodo.after(mark);
  });
}
```